

TCS302: Data Structures with C

Unit 1, Lectures 3 & 4

Pointers, Structures, Dynamic Memory and Algorithm Complexity

How C memory tools influence program
performance and design.

Manoj Kaushik
Department of Computer Science and
Engineering



Where Are We in Unit 1?

We now connect C memory organization with algorithm performance.

Lecture 1
Basic terminology

Lecture 2
Operations + ADT

Lectures 3 & 4
**Memory +
complexity**

- Pointers access memory through addresses.
- Structures group related fields into records.
- Dynamic allocation creates data at run time.

Question: Why must a data-structure programmer understand memory addresses?

Learning Outcomes

By the end of this combined lecture, students should be able to:

1

Use address and dereference operators

3

Allocate and release heap memory

2

Build and access structures

4

Analyse time and space growth

Class rule: draw the memory picture before tracing pointer code.

Pointer Vocabulary

A pointer is a typed variable that stores a memory address.

Address (&)

Pointer (*)

Dereference

NULL

Pointer arithmetic

Structure pointer

**Dynamic
pointer**

- Type tells C how many bytes to read and how far p+1 moves.
 - An uninitialized pointer must never be dereferenced.
-

Declare and Initialize a Pointer

Always give a pointer a valid address or NULL before using it.

```
int x = 25;  
int *p = &x;
```

x	25	p	&x	1000
---	----	---	----	------

- &x obtains the address of x.
- p stores that address.
- *p accesses the value at that address.

Ask students: If p stores 1000, what does *p produce?

Dereferencing Reads or Changes the Original Value

The operator `*` follows an address to the stored value.



```
int x = 10;  
int *p = &x;  
*p = 35;
```

- `p` gives the address of `x`.
- `*p` reads the current value of `x`.
- Assigning to `*p` changes `x` itself.

Quick check: After `*p = 35`, what value is stored in `x`?

Pointer Arithmetic with Arrays

For an int pointer, $p+1$ advances to the next integer element.



p points here

- The array name acts like the address of its first element.
- $*(p+2)$ accesses the third element.
- Movement depends on the pointed-to type.

```
printf("%d", *(p + 2));
```

Three Ways Pointers Work in C

The same address idea supports reading, modifying and linking data.

Read

- `int value = *p;`
- obtains pointed value
- no change to memory

Modify

- `*p = 35;`
- updates original value
- used in functions

Link

- `node->next = q;`
- connects structures
- builds dynamic lists

Real example: `swap(&a, &b)` lets a function change both variables.

Structure Access: Object versus Pointer

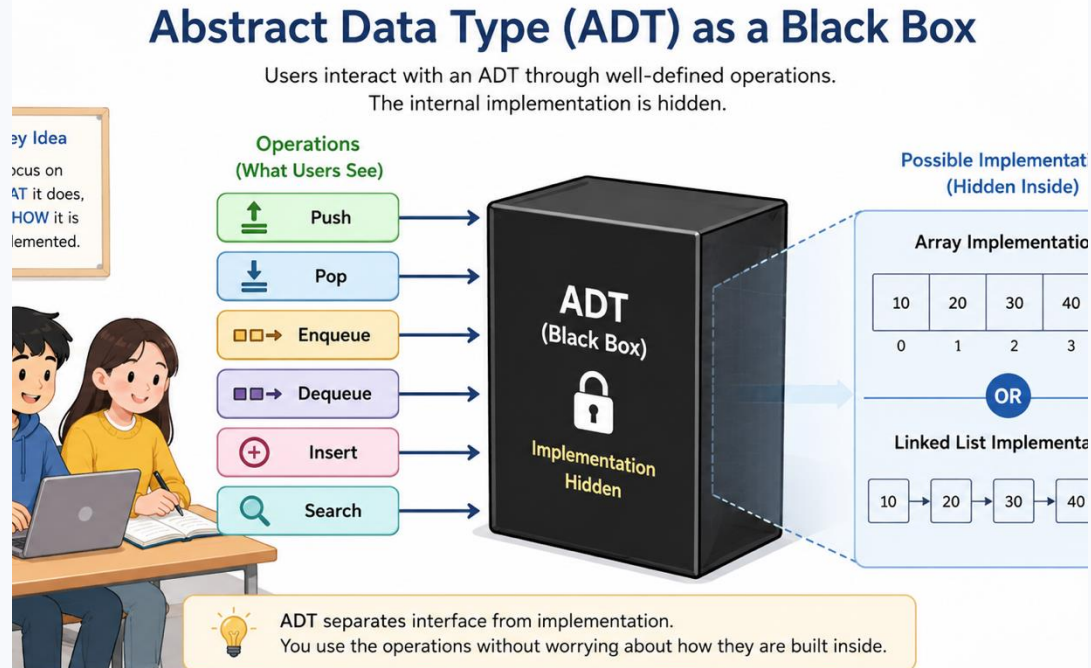
Choose . for an object and -> for a pointer to an object.

Expression	Direct object	Structure pointer
Access name	s.name	ps->name
Access CGPA	s.cgpa	ps->cgpa
Equivalent form	(*ps).cgpa	ps->cgpa

Key idea: ps->field is a readable form of (*ps).field.

Dynamic Memory: Main Idea

Heap memory is requested while the program is running.



- Use malloc or calloc when size is known only at run time.
- The returned address is stored in a pointer.
- The program must release the block using free.

Analogy: Borrow a memory block, use it carefully, then return it.

Dynamic Memory = Request + Check + Release

A safe allocation sequence has three essential responsibilities.

Request
malloc / calloc

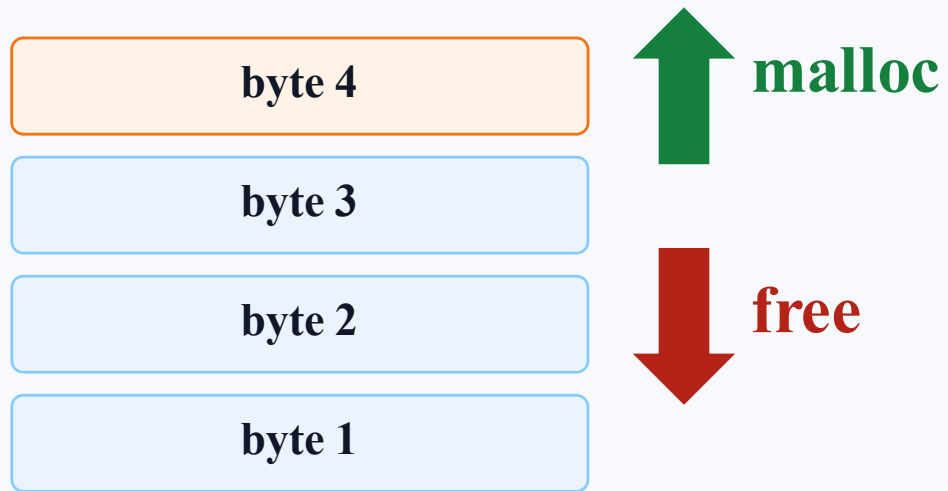
Check
pointer != NULL

Release
free(pointer)

Example: `int *a = malloc(n * sizeof *a);`

Heap Allocation in C

malloc reserves bytes; free returns the same block to the heap.

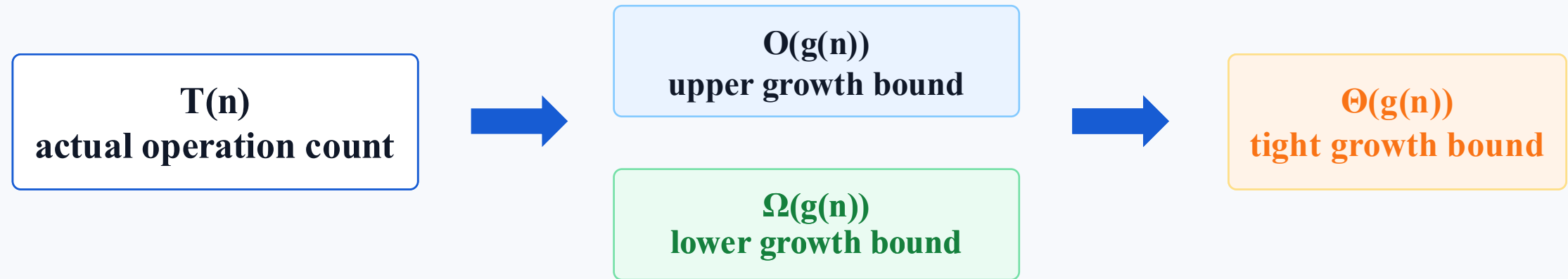


- `int *p = malloc(4 * sizeof *p);`
- `check p != NULL`
- `use p[0] ... p[3]`
- `free(p); p = NULL;`

Question: Why should p not be used after free(p)?

Growth Notation Describes Algorithm Performance

Focus on how cost changes when input size n becomes large.



Key idea: $3n^2 + 5n + 8$ belongs to $\Theta(n^2)$.

Analyse Loops by Counting Repeated Work

Consecutive blocks add their costs; nested loops multiply them.

```
for (int i = 0; i < n; i++) {  
    sum += a[i];  
}
```

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        count++;
```

- First loop performs $\Theta(n)$ work.
- Nested loops perform $\Theta(n^2)$ work.
- Keep the dominant term for large n .

Space question: Does either loop allocate an n -element auxiliary array?

Classroom Analysis: Find Time and Extra Space

Trace the code first; then state complexity using n .

Problem: Count how many pairs are tested in the nested loops.

- How many times does the outer loop run?
- How many times does the inner statement run?
- What is the dominant term?
- Is extra storage dependent on n ?

```
for (i=0; i<n; i++)  
  for (j=0; j<n; j++)  
    count++;
```

Time: $\Theta(n^2)$
Extra space: $\Theta(1)$

Time–Space Trade-off

A faster operation may require additional memory.

Using less memory

- Linear search: $O(n)$ time
- Extra space: $O(1)$
- No preprocessing

Using more memory

- Hash table lookup: average $O(1)$
- Extra space: $O(n)$
- Requires table construction

Decision: choose according to input size, memory limit and frequent operations.